SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)
(Established under section 3 of the UGC Act 1956)
Re - accredited by NAAC with 'A' Grade
Founder: Prof. Dr. S. B. Mujumdar, M.Sc.,Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Database Management Systems Lab

Project Phase: 2

Group Members:

Sr.No.	Name	PRN	Branch	Division
1	Mihir Hebalkar	22070122120	CS	B2
2	Sachin Mhetre	22070122119	CS	B2
3	Parimal Kulkarni	22070122138	CS	B2
4	Onkar Mendhapurkar	22070122135	CS	B2

Under the Guidance of
Dr. Pooja Bagane

Department of Computer Science
Symbiosis Institute of Technology, Pune

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)
(Established under section 3 of the UGC Act 1956)
Re - accredited by NAAC with 'A' Grade
Founder: Prof. Dr. S. B. Mujumdar, M.Sc.,Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

Title of the project:

Personal Finance Management System

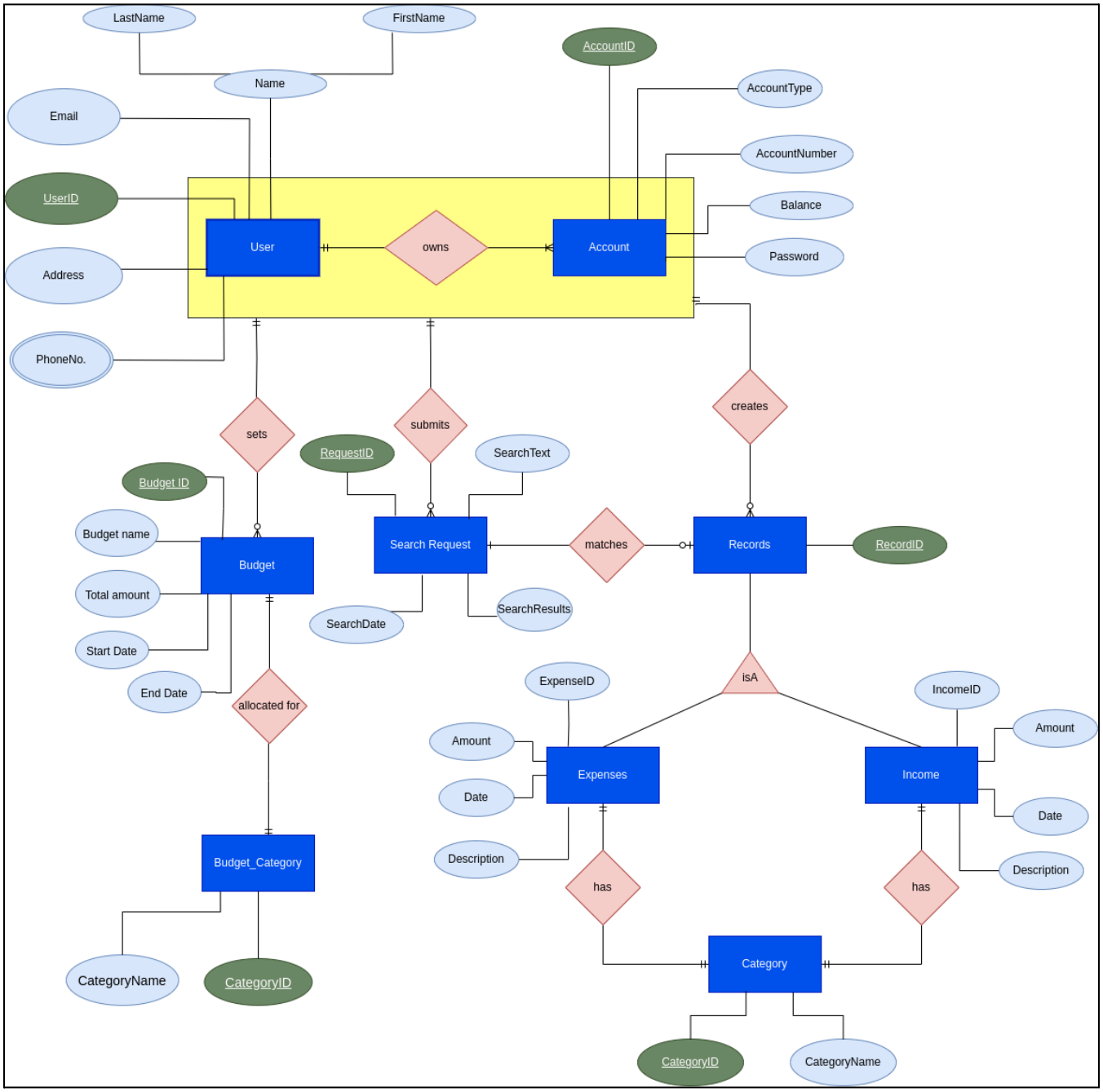
About the project:

The Personal Finance Management System (PFMS) is a sophisticated application designed to empower users in managing their finances with precision and ease. Developed using Java Swing for the user interface and MySQL for robust database management, PFMS offers a comprehensive suite of features tailored to financial tracking and analysis. Through PFMS, users can register and create multiple accounts, each meticulously categorized to represent different financial aspects such as savings, checking, and investments. Income and expenses can be recorded seamlessly, with detailed information including dates, sources, categories, and amounts, ensuring a granular understanding of financial activities.

Furthermore, PFMS facilitates effective budgeting and goal setting by allowing users to establish budgets for various expense categories and define target amounts to achieve specific financial milestones. The system's architecture prioritizes data integrity and security, employing foreign key constraints and triggers to maintain consistency within the database while ensuring that only authorized users can access and modify financial data. Additionally, PFMS offers visualization tools such as charts to analyze monthly financial data, empowering users to make informed decisions and gain insights into their financial habits and trends. With its scalable and flexible design, PFMS stands as a reliable companion for individuals seeking to take control of their financial well-being.

Addressed paint points:
The pain point addressed by the Personal Finance Management System (PFMS) lies in the complexity and disorganization inherent in manual financial tracking. Many individuals struggle to manage multiple accounts, track various income sources, and categorize expenses efficiently. PFMS offers a solution by providing a user-friendly interface and robust database management, streamlining the process of recording and organizing financial data. By centralizing income, expenses, and budgets within a structured framework, PFMS eliminates the frustration of scattered financial records and enables users to gain clarity and control over their finances effectively.

ER diagram:


LastName
FirstName
Name
Email
UserID
Address
PhoneNo.
User
owns
Account
AccountID
AccountType
AccountNumber
Balance
Password
sets
submits
creates
RequestID
SearchText
Budget ID
Budget name
Budget
Total amount
Start Date
End Date
Search Request
matches
Records
RecordID
SearchDate
SearchResults
allocated for
isA
ExpenseID
IncomeID
Amount
Expenses
Date
Description
Income
Amount
Date
Description
Budget_Category
has
has
Category
CategoryName
CategoryID
CategoryID
CategoryName

Anomalies:
A database anomaly is a fault within a database, which can occur because of poor planning or when everything is stored in a flat database. These anomalies naturally occur and result in data that does not match the real-world the database purports to represent.

Anomalies occurring in our database:

1) **1NF:** *Insert Anomaly*: In the *Budget* table, having multiple category names within a single cell (multi-valued attribute CategoryName) violates the first normal form (1NF) > *BudgetID=3*. This means if we want to insert a new *budget* entry with multiple categories, we need to repeat all other attribute values for each category, leading to redundant data.
Update Anomaly: Updating a category name for a specific *BudgetID* would require updating multiple rows if the *BudgetID* has multiple categories associated with it, leading to inconsistencies.
Delete Anomaly: Deleting a specific category associated with a *BudgetID* would lead to loss of information about other categories if they are stored within the same cell, potentially breaking the budget entry.

BudgetID	BudgetName	TotalAmount	StartDate	EndDate	AccountID	BudgetCategoryID	CategoryName
1	Monthly Expenses	3000.00	2024-04-01	2024-04-30	1	101	Utilities
2	Groceries	500.00	2024-04-01	2024-04-30	1	102	Food
3	Entertainment	200.00	2024-04-01	2024-04-30	2	103	Movies
3	Entertainment	200.00	2024-04-01	2024-04-30	2	103	Movies
4	Travel	1000.00	2024-04-01	2024-04-30	2	104	Vacation
5	Savings	1500.00	2024-04-01	2024-04-30	3	105	Emergency Fund

2) **2NF:** Insert Anomaly: To insert a new user, we must provide all attributes including *UserID*, which may not always be available or relevant at the time of insertion.
Update Anomaly: Updating information about a user requires updating potentially multiple rows if the user has multiple accounts, leading to inconsistency.
Delete Anomaly: Deleting a user might result in losing information about the associated accounts, leading to data inconsistency.

AccountID	AccountType	Balance	AccountNumber	Password	UserID	Fname	Lname	Username	Email
1	Savings	5000.00	1234567890	pass123	101	John	Doe	john.doe	john@example.com
2	Checking	2500.00	0987654321	pass456	102	Jane	Smith	jane.smith	jane@example.com
3	Savings	10000.00	2468135790	pass789	103	Michael	Johnson	michael.johnson	michael@example.com
4	Investment	15000.00	1357924680	passabc	104	Emily	Brown	emily.brown	emily@example.com
5	Checking	3500.00	9876543210	passxyz	105	David	Wilson	david.wilson	david@example.com

3) **3NF:** Insert Anomaly: Inserting a new expense entry requires providing both CategoryID and CategoryName, leading to potential data redundancy and inconsistency.
Update Anomaly: Updating a category's name or description would require updating potentially multiple rows if the category is associated with multiple expenses, leading to inconsistency.
Delete Anomaly: Deleting a category might result in losing information about expenses associated with that category, leading to data inconsistency.

RecordID	ExpenseID	Amount	Date	Description	CategoryID	CategoryName
1	101	50.00	2024-04-01	Groceries	1	Food
2	102	30.25	2024-04-02	Gasoline	2	Transportation
3	103	100.00	2024-04-03	Dinner with friends	3	Entertainment
4	104	20.99	2024-04-04	Office supplies	4	Business
5	105	75.50	2024-04-05	Restaurant	1	Food

Functional dependencies:
Functional dependency refers to a relationship that is present between attributes of any table that are dependent on each other. It helps in avoiding data redundancy and getting to know more about bad designs.

Functional dependencies of schema occurring in our code:

1. Users Table:
   - UserID -> Username, Fname, Lname, Email
   - Username -> UserID
   - Email -> UserID

2. PhoneNo Table:
   - UserID -> PhoneNo

3. Account Table:
   - AccountID -> AccountType, Balance, AccountNumber, Password, UserID
   - UserID -> AccountID

4. Owns Table:
   - (UserID, AccountID) -> UserID, AccountID

5. Budget Table:
   - BudgetID -> BudgetName, TotalAmount, StartDate, EndDate, AccountID, BudgetCategoryID
   - AccountID -> BudgetID

- BudgetCategoryID -> BudgetID

6. Budget_Category Table:
   - CategoryID -> CategoryName

7. Records Table:
   - RecordID -> AccountID

8. Search_Request Table:
   - RequestID -> SearchText, SearchDate, SearchResults, AccountID, RecordID

9. Expenses Table:
   - (RecordID, ExpenseID) -> Amount, Date, Description, CategoryID
10. Income Table:
   - (RecordID, IncomeID) -> Amount, Date, Description, CategoryID

11. Category Table:
   - CategoryID -> CategoryName

Normalization:
A normalization procedure, which combines and splits tables, is usually sufficient to remove this. By normalizing the database, we reduce the likelihood of creating tables that generate anomalies. Normalization is necessary if you do not do it then the overall integrity of the data stored in the database will eventually degrade. Specifically, this is due to data anomalies.

Types of normalization

1. Unnormalized form or UNF:
It is the simplest database model also known as non-first normal form (NF2).A UNF model will suffer problems like data redundancy thus it lacks the efficiency of database normalization.
2. First Normal Form or 1NF:
A relation is in first normal form only if the relational table doesn't contain any multivalued attribute but contains only single-valued attributes.
3. Second Normal Form or 2NF:
A relation is in second normal form if:

• It is in first normal form or 1NF
• It doesn't contain any partial dependencies. (It shouldn't have any non-prime attribute which is functionally dependent on any proper subset of the candidate key of the relation.).

4. Third Normal Form or 3NF:

Let R be the relational schema, X->Y any non-trivial functional dependency over R is in 3NF if:
• R should be in 2NF
• X should be candidate key or superkey, or
• Y should be prime attribute

Normalization occurring in our code:

1) **1NF:** In the unnormalized version of the *Budget* table, the presence of the multi-valued attribute *CategoryName* violates the first normal form (1NF) by allowing multiple values within a single cell. To adhere to 1NF, we create a separate table, *Budget_Category*, to store individual category names. This normalization ensures each attribute in the Budget table holds atomic values, promoting data integrity and efficient querying.

BudgetID	BudgetName	TotalAmount	StartDate	EndDate	AccountID	BudgetCategoryID	CategoryName
1	Monthly Expenses	3000.00	2024-04-01	2024-04-30	1	101	Utilities
2	Groceries	500.00	2024-04-01	2024-04-30	1	102	Food
3	Entertainment	200.00	2024-04-01	2024-04-30	2	103	Movies
3	Entertainment	200.00	2024-04-01	2024-04-30	2	103	Movies
4	Travel	1000.00	2024-04-01	2024-04-30	2	104	Vacation
5	Savings	1500.00	2024-04-01	2024-04-30	3	105	Emergency Fund

Creating a separate table for *Budget_Category.*

BudgetID	BudgetName	TotalAmount	StartDate	EndDate	AccountID	BudgetCategoryID
1	Monthly Expenses	3000.00	2024-04-01	2024-04-30	1	101
2	Groceries	500.00	2024-04-01	2024-04-30	1	102
3	Entertainment	200.00	2024-04-01	2024-04-30	2	103
4	Travel	1000.00	2024-04-01	2024-04-30	2	104
5	Savings	1500.00	2024-04-01	2024-04-30	3	105

CategoryID	CategoryName
102	Food
103	Movies
104	Vacation
105	Emergency Fund

2) **2NF:** The attributes *Fname, Lname, Username, and Email* are functionally dependent only on *UserID*, but not on the entire primary key (which includes *AccountID*). They describe the user associated with the account, not the account itself. This is a partial dependency because these attributes depend on part of the primary key.

AccountID	AccountType	Balance	AccountNumber	Password	UserID	Fname	Lname	Username	Email
1	Savings	5000.00	1234567890	pass123	101	John	Doe	john.doe	john@example.com
2	Checking	2500.00	0987654321	pass456	102	Jane	Smith	jane.smith	jane@example.com
3	Savings	10000.00	2468135790	pass789	103	Michael	Johnson	michael.johnson	michael@example.com
4	Investment	15000.00	1357924680	passabc	104	Emily	Brown	emily.brown	emily@example.com
5	Checking	3500.00	9876543210	passxyz	105	David	Wilson	david.wilson	david@example.com

To remove partial dependency and achieve 2NF, we need to separate *user*-related attributes into a separate table.

UserID	Fname	Lname	Username	Email
1	John	Doe	johndoe	john.doe@example.com
2	Jane	Smith	janesmith	jane.smith@example.com
3	Michael	Johnson	michaeljohnson	michael.johnson@example.com
4	Emily	Brown	emilybrown	emily.brown@example.com
5	William	Martinez	williammartinez	william.martinez@example.com
6	Olivia	Taylor	oliviataylor	olivia.taylor@example.com
7	James	Anderson	jamesanderson	james.anderson@example.com
8	Sophia	Thomas	sophiathomas	sophia.thomas@example.com

AccountID	AccountType	Balance	AccountNumber	Password	UserID
1	Savings	1000.00	101	password1	1
2	Checking	500.00	102	password2	2
3	Savings	1500.00	135	password3	3
4	Checking	200.00	105	password4	4
5	Savings	800.00	108	password5	5
6	Checking	700.00	117	password6	6
7	Savings	1200.00	118	password7	7
8	Checking	300.00	109	password8	8

3) **3NF:** The attributes *CategoryName* and *CategoryDescription* are functionally dependent on *CategoryID*, but they are indirectly dependent on the primary key through *RecordID*. These attributes describe the category associated with the expense, which is determined by *CategoryID, not RecordID*. Therefore, it's a transitive dependency because it depends indirectly on the primary key through another non-key attribute.

RecordID	ExpenseID	Amount	Date	Description	CategoryID	CategoryName
1	101	50.00	2024-04-01	Groceries	1	Food
2	102	30.25	2024-04-02	Gasoline	2	Transportation
3	103	100.00	2024-04-03	Dinner with friends	3	Entertainment
4	104	20.99	2024-04-04	Office supplies	4	Business
5	105	75.50	2024-04-05	Restaurant	1	Food

To remove the transitive dependency and achieve 3NF, we need to separate the attributes related to *categories* into a separate table.

RecordID	ExpenseID	Amount	Date	Description	CategoryID
3	3	100.00	2024-01-10	Electricity bill	4
4	4	30.00	2024-01-20	Bus fare	5
5	5	50.00	2024-01-25	Books	6
6	6	80.00	2024-01-08	Doctor visit	7
7	7	200.00	2024-01-14	Flight tickets	8
8	8	60.00	2024-01-16	T-shirts	9

CategoryID	CategoryName
1	Food
2	Salary
3	Rent
4	Utilities
5	Transportation

What is MySQL:
MySQL is a relational database management system (RDBMS) developed by Oracle that is based on structured query language (SQL).

Implementation of the tables in MySQL:

```
create database dbmsReport;

use dbmsReport;
```

```sql
CREATE TABLE User (
    UserID INT PRIMARY KEY,
    Fname VARCHAR(20),
    Lname VARCHAR(20),
    Username VARCHAR(50),
    Email VARCHAR(100)
);


CREATE TABLE PhoneNo (
    UserID INT,
    PhoneNo VARCHAR(12),
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);

CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    AccountType VARCHAR(50),
    Balance DECIMAL(10,2),
    AccountNumber VARCHAR(20),
    Password VARCHAR(50),
    UserID INT,
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);

CREATE TABLE Owns (
    UserID INT,
    AccountID INT,
    FOREIGN KEY (UserID) REFERENCES User(UserID),
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
);

CREATE TABLE Budget (
    BudgetID INT PRIMARY KEY,
    BudgetName VARCHAR(100),
```

```
    TotalAmount DECIMAL(10,2),
    StartDate DATE,
    EndDate DATE,
    AccountID INT,
    BudgetCategoryID INT,
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
);

CREATE TABLE Budget_Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(100)
);

CREATE TABLE Records (
    RecordID INT PRIMARY KEY,
    AccountID INT,
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
);

CREATE TABLE Search_Request (
    RequestID INT PRIMARY KEY,
    SearchText VARCHAR(100),
    SearchDate DATE,
    SearchResults INT,
    AccountID INT,
    RecordID INT,
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID),
    FOREIGN KEY (RecordID) REFERENCES Records(RecordID)
);

CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(100)
);
```

```
CREATE TABLE Expenses (
    RecordID INT,
    ExpenseID INT,
    Amount DECIMAL(10,2),
    Date DATE,
    Description VARCHAR(100),
    CategoryID INT,
    FOREIGN KEY (RecordID) REFERENCES Records(RecordID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE Income (
    RecordID INT,
    IncomeID INT,
    Amount DECIMAL(10,2),
    Date DATE,
    Description VARCHAR(100),
    CategoryID INT,
    FOREIGN KEY (RecordID) REFERENCES Records(RecordID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

delimiter //
create trigger afterincomeinsert
after insert
on Income
for each row
begin
    declare newbalance DECIMAL(10,2);
    declare updateid INT;
    SET updateid = (SELECT accountid FROM records WHERE recordid = NEW.recordid);
    SET newbalance = (SELECT balance + NEW.amount FROM account WHERE accountid =
updateid);
    update account SET balance = newbalance WHERE accountid = updateid;
end;
//
```

```
delimiter ;

delimiter //
create trigger afterExpenseinsert
after insert
on Expenses
for each row
begin
    declare newbalance DECIMAL(10,2);
    declare updateid INT;
    SET updateid = (SELECT accountid FROM records WHERE recordid = NEW.recordid);
    SET newbalance = (SELECT balance - NEW.amount FROM account WHERE accountid =
updateid);
    update account SET balance = newbalance WHERE accountid = updateid;
end;
//
delimiter ;


delimiter //
create procedure showAccountDetails (
 IN in_username varchar(50)
 )
begin
      select u.Username,a.accountnumber, a.accountid, a.accounttype, a.balance
    from Account a
    join User u on a.UserID = u.UserID
    where u.Username = in_username;
end;
//
delimiter ;



delimiter //
create procedure changePassword (
```

```
      in_username varchar(50),
    in_password varchar(50),
    new_password varchar(50)
)
begin
      declare existing_pass varchar(50);

    select password into existing_pass from account where userid = (
            select userid from user where username = in_username
      );
    if existing_pass is not null and existing_pass = in_password then
            UPDATE Account SET password = new_password where userid = (
            select userid from user where username = in_username
      );
    end if;
end;
//
delimiter ;

call changePassword('johndoe','password1','newpassword');

delimiter //
create function currentBal(in_username varchar(50))
returns decimal(10,2)
reads sql data
begin
      declare cur_bal decimal(10,2);

    select sum(i.amount) - sum(e.amount) into cur_bal
    from account a
    join records r on a.accountid = r.accountid
    join income i on r.recordid = i.recordid
    join expenses e on r.recordid = e.recordid
    join user u on u.userid = a.userid
      where u.username = in_username;
```

```
      return cur_bal;
end;
//
delimiter ;

select currentBal('johndoe') "Balance";

delimiter //
create function expenseForCategory(in_category varchar(20))
returns decimal(10,2)
reads sql data
begin
      declare totalExpense decimal(10,2);
      select sum(amount) into totalExpense from expenses
    where categoryid = (select CategoryID from Category where CategoryName =
in_category);
    return totalExpense;
end;
//
delimiter ;

select expenseForCategory('Food') as FoodExpense;

INSERT INTO User (UserID, Fname, Lname, Username, Email)
VALUES (1, 'John', 'Doe', 'johndoe', 'john.doe@example.com'),
       (2, 'Jane', 'Smith', 'janesmith', 'jane.smith@example.com'),
       (3, 'Michael', 'Johnson', 'michaeljohnson', 'michael.johnson@example.com'),
       (4, 'Emily', 'Brown', 'emilybrown', 'emily.brown@example.com'),
       (5, 'William', 'Martinez', 'williammartinez', 'william.martinez@example.com'),
       (6, 'Olivia', 'Taylor', 'oliviataylor', 'olivia.taylor@example.com'),
       (7, 'James', 'Anderson', 'jamesanderson', 'james.anderson@example.com'),
       (8, 'Sophia', 'Thomas', 'sophiathomas', 'sophia.thomas@example.com'),
       (9, 'David', 'Hernandez', 'davidhernandez', 'david.hernandez@example.com'),
       (10, 'Emma', 'Garcia', 'emmagarcia', 'emma.garcia@example.com');
```

```sql
INSERT INTO PhoneNo (UserID, PhoneNo)
VALUES (1, '9315462104'),
       (2, '9876543210'),
       (3, '7152467904'),
       (4, '9845064805'),
       (5, '8094505456'),
       (6, '8894770245'),
       (7, '9171819202'),
       (8, '7222324252'),
       (9, '8272829301'),
       (10, '7732333536');


INSERT INTO Account (AccountID, AccountType, Balance, AccountNumber, Password, UserID)
VALUES (1, 'Savings', 1000.00, '101', 'password1', 1),
       (2, 'Checking', 500.00, '102', 'password2', 2),
       (3, 'Savings', 1500.00, '135', 'password3', 3),
       (4, 'Checking', 200.00, '105', 'password4', 4),
       (5, 'Savings', 800.00, '108', 'password5', 5),
       (6, 'Checking', 700.00, '117', 'password6', 6),
       (7, 'Savings', 1200.00, '118', 'password7', 7),
       (8, 'Checking', 300.00, '109', 'password8', 8),
       (9, 'Savings', 2000.00, '120', 'password9', 9),
       (10, 'Checking', 400.00, '138', 'password10', 10);

INSERT INTO Owns (UserID, AccountID)
VALUES (1, 1),
       (2, 2),
       (3, 3),
       (4, 4),
       (5, 5),
       (6, 6),
       (7, 7),
       (8, 8),
       (9, 9),
       (10, 10);
```

```sql
INSERT INTO Budget_Category (CategoryID, CategoryName)
VALUES (1, 'Groceries'),
       (2, 'Entertainment'),
       (3, 'Utilities'),
       (4, 'Transportation'),
       (5, 'Education'),
       (6, 'Healthcare'),
       (7, 'Travel'),
       (8, 'Clothing'),
       (9, 'Dining Out'),
       (10, 'Gifts');


INSERT INTO Records (RecordID, AccountID)
VALUES (1, 1),
       (2, 2),
       (3, 3),
       (4, 4),
       (5, 5),
       (6, 6),
       (7, 7),
       (8, 8),
       (9, 9),
       (10, 10);


INSERT INTO Category (CategoryID, CategoryName)
VALUES (1, 'Food'),
       (2, 'Salary'),
       (3, 'Rent'),
       (4, 'Utilities'),
       (5, 'Transportation'),
       (6, 'Education'),
       (7, 'Healthcare'),
```

```
       (8, 'Travel'),
       (9, 'Clothing'),
       (10, 'Entertainment');


INSERT INTO Expenses (RecordID, ExpenseID, Amount, Date, Description, CategoryID)
VALUES (1, 1, 50.00, '2024-01-05', 'Groceries', 1),
       (2, 2, 200.00, '2024-01-15', 'Dinner with friends', 10),
       (3, 3, 100.00, '2024-01-10', 'Electricity bill', 4),
       (4, 4, 30.00, '2024-01-20', 'Bus fare', 5),
       (5, 5, 50.00, '2024-01-25', 'Books', 6),
       (6, 6, 80.00, '2024-01-08', 'Doctor visit', 7),
       (7, 7, 200.00, '2024-01-14', 'Flight tickets', 8),
       (8, 8, 60.00, '2024-01-16', 'T-shirts', 9),
       (9, 9, 70.00, '2024-01-18', 'Movie tickets', 10),
       (10, 10, 40.00, '2024-01-30', 'Gift for friend', 10);


INSERT INTO Income (RecordID, IncomeID, Amount, Date, Description, CategoryID)
VALUES (1, 1, 2000.00, '2024-01-10', 'Monthly salary', 2),
       (2, 2, 500.00, '2024-01-20', 'Freelance work', 2),
       (3, 3, 1500.00, '2024-01-05', 'Rent payment', 3),
       (4, 4, 300.00, '2024-01-15', 'Utility refund', 4),
       (5, 5, 1000.00, '2024-01-25', 'Transportation reimbursement', 5),
       (6, 6, 800.00, '2024-01-08', 'Scholarship', 6),
       (7, 7, 500.00, '2024-01-14', 'Insurance claim', 7),
       (8, 8, 700.00, '2024-01-16', 'Travel allowance', 8),
       (9, 9, 400.00, '2024-01-18', 'Clothing allowance', 9),
       (10, 10, 300.00, '2024-01-30', 'Gift from family', 10);

INSERT INTO Budget (BudgetID, BudgetName, TotalAmount, StartDate, EndDate, AccountID,
BudgetCategoryID)
VALUES
      (1, 'Monthly Expenses', 1500.00, '2024-04-01', '2024-04-30', 1, 1),
      (2, 'Entertainment', 200.00, '2024-04-01', '2024-04-30', 1, 2),
      (3, 'Utilities', 300.00, '2024-04-01', '2024-04-30', 2, 3),
```

```
      (4, 'Transportation', 100.00, '2024-04-01', '2024-04-30', 2, 4),
      (5, 'Education', 500.00, '2024-04-01', '2024-04-30', 3, 5);


INSERT INTO Search_Request (RequestID, SearchText, SearchDate, SearchResults,
AccountID, RecordID)
VALUES (1, 'Groceries', '2024-04-26', 5, 1, 1);
```

Table outputs:

1. User

#	UserID	Fname	Lname	Username	Email
1	1	John	Doe	johndoe	john.doe@example.com
2	2	Jane	Smith	janesmith	jane.smith@example.com
3	3	Michael	Johnson	michaeljohnson	michael.johnson@example.com
4	4	Emily	Brown	emilybrown	emily.brown@example.com
5	5	William	Martinez	williammartinez	william.martinez@example.com
6	6	Olivia	Taylor	oliviataylor	olivia.taylor@example.com
7	7	James	Anderson	jamesanderson	james.anderson@example.com
8	8	Sophia	Thomas	sophiathomas	sophia.thomas@example.com
9	9	David	Hernandez	davidhernandez	david.hernandez@example.com
10	10	Emma	Garcia	emmagarcia	emma.garcia@example.com

2. Phone

#	UserID	PhoneNo
1	1	9315462104
2	2	9876543210
3	3	7152467904
4	4	9845064805
5	5	8094505456
6	6	8894770245
7	7	9171819202
8	8	7222324252
9	9	8272829301
10	10	7732333536

3. Account

#	AccountID	AccountType	Balance	AccountNumbe	Password	UserID
1	1	Savings	1000.00	101	password1	1
2	2	Checking	500.00	102	password2	2
3	3	Savings	1500.00	135	password3	3
4	4	Checking	200.00	105	password4	4
5	5	Savings	800.00	108	password5	5
6	6	Checking	700.00	117	password6	6
7	7	Savings	1200.00	118	password7	7
8	8	Checking	300.00	109	password8	8
9	9	Savings	2000.00	120	password9	9
10	10	Checking	400.00	138	password10	10

4. Owns

#	UserID	AccountID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10	10	10

5. Budget

#	BudgetID	BudgetName	TotalAmount	StartDate	EndDate	AccountID	BudgetCategoryII
1	1	Monthly Expen…	1500.00	2024-04-01	2024-04…	1	1
2	2	Entertainment	200.00	2024-04-01	2024-04…	1	2
3	3	Utilities	300.00	2024-04-01	2024-04…	2	3
4	4	Transportation	100.00	2024-04-01	2024-04…	2	4
5	5	Education	500.00	2024-04-01	2024-04…	3	5

6. Budget Category

#	CategoryID	CategoryName
1	1	Groceries
2	2	Entertainment
3	3	Utilities
4	4	Transportation
5	5	Education
6	6	Healthcare
7	7	Travel
8	8	Clothing
9	9	Dining Out
10	10	Gifts

7. Records

#	RecordID	AccountID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10	10	10

8. Search Request

#	RequestID	SearchText	SearchDate	SearchResults	AccountID	RecordID
1	1	Groceries	2024-04-26	5	1	1

9. Category

#	CategoryID	CategoryName
1	1	Food
2	2	Salary
3	3	Rent
4	4	Utilities
5	5	Transportation
6	6	Education
7	7	Healthcare
8	8	Travel
9	9	Clothing
10	10	Entertainment

10. Expense

#	RecordID	ExpenseID	Amount	Date	Description	CategoryID
1	1	1	50.00	2024-01-05	Groceries	1
2	2	2	200.00	2024-01-15	Dinner with friends	10
3	3	3	100.00	2024-01-10	Electricity bill	4
4	4	4	30.00	2024-01-20	Bus fare	5
5	5	5	50.00	2024-01-25	Books	6
6	6	6	80.00	2024-01-08	Doctor visit	7
7	7	7	200.00	2024-01-14	Flight tickets	8
8	8	8	60.00	2024-01-16	T-shirts	9
9	9	9	70.00	2024-01-18	Movie tickets	10
10	10	10	40.00	2024-01-30	Gift for friend	10

11. Income

#	RecordID	IncomeID	Amount	Date	Description	CategoryID
1	1	1	2000.00	2024-01-10	Monthly salary	2
2	2	2	500.00	2024-01-20	Freelance work	2
3	3	3	1500.00	2024-01-05	Rent payment	3
4	4	4	300.00	2024-01-15	Utility refund	4
5	5	5	1000.00	2024-01-25	Transportation reimbursement	5
6	6	6	800.00	2024-01-08	Scholarship	6
7	7	7	500.00	2024-01-14	Insurance claim	7
8	8	8	700.00	2024-01-16	Travel allowance	8
9	9	9	400.00	2024-01-18	Clothing allowance	9
10	10	10	300.00	2024-01-30	Gift from family	10

Queries implemented:

Query 1: To display account information with balance.

```
select a.accountid, concat(u.fname, " ", u.lname) "Name", a.balance from account a join user u on a.userid = u.userid;
```

Result Grid Filter Rows:

accountid	Name	balance
1	John Doe	1000.00
2	Jane Smith	500.00
3	Michael Johnson	1500.00
4	Emily Brown	200.00
5	William Martinez	800.00
6	Olivia Taylor	700.00
7	James Anderson	1200.00
8	Sophia Thomas	300.00
9	David Hernandez	2000.00
10	Emma Garcia	400.00

Query 2: To display total expense of user category-wise.

```
select r.accountid, c.categoryname "ExpenseCategory", sum(e.amount) "Expense"
from expenses e join records r on r.recordid = e.recordid join category c on c.categoryid = e.categoryid
group by categoryname, r.accountid;
```

Result Grid | Filter Rows:

accountid	ExpenseCategory	Expense
1	Food	50.00
2	Entertainment	200.00
3	Utilities	100.00
4	Transportation	30.00
5	Education	50.00
6	Healthcare	80.00
7	Travel	200.00
8	Clothing	60.00
9	Entertainment	70.00
10	Entertainment	40.00

Query 3: To display current balance of the user.

```
select r.accountid,i.amount - e.amount "Current Balance"
from records r join income i on i.recordid = r.recordid join expenses e on e.recordid = r.recordid;
```

Result Grid | Filter Rows:

accountid	Current Balance
1	1950.00
2	300.00
3	1400.00
4	270.00
5	950.00
6	720.00
7	300.00
8	640.00
9	330.00
10	260.00

Query 4: To show total budget allocated by user category-wise.

```
select b.accountid,bc.CategoryName as BudgetCategory, sum(b.TotalAmount) as TotalBudget
from budget b join budget_category bc on b.budgetcategoryid = bc.categoryid
group by b.accountid,bc.categoryname;
```

Result Grid | Filter Rows:

accountid	BudgetCategory	TotalBudget
1	Groceries	1500.00
1	Entertainment	200.00
2	Utilities	300.00
2	Transportation	100.00
3	Education	500.00

Query 5: To display month wise income and expense comparison for every account.

```
select r.AccountID,monthname(e.Date) AS Month,sum(i.Amount) as TotalIncome, sum(e.Amount) as TotalExpenses
from Records r
left join Income i on r.recordID = i.recordID
left join Expenses e on r.RecordID = e.RecordID
group by r.accountID, monthname(e.Date);
```

Result Grid | Filter Rows: | Export:

AccountID	Month	TotalIncome	TotalExpenses
1	January	2000.00	50.00
2	January	500.00	200.00
3	January	1500.00	100.00
4	January	300.00	30.00
5	January	1000.00	50.00
6	January	800.00	80.00
7	January	500.00	200.00
8	January	700.00	60.00
9	January	400.00	70.00
10	January	300.00	40.00

Function:
A function is a set of SQL statements, that you can pass parameters into, and they perform some operation and return a single value.

Functions:

1. Function to get current Balance of a user.

```
delimiter //
create function currentBal(in_username varchar(50))
returns decimal(10,2)
reads sql data
begin
    declare cur_bal decimal(10,2);

    select sum(i.amount) - sum(e.amount) into cur_bal
    from account a
    join records r on a.accountid = r.accountid
    join income i on r.recordid = i.recordid
    join expenses e on r.recordid = e.recordid
    join user u on u.userid = a.userid
    where u.username = in_username;

    return cur_bal;
end;
//
delimiter ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:00:18	create function currentBal(in_username varchar(50)) returns decimal(10,2) reads sql data begin declare cur_bal d...	0 row(s) affected	0.015 sec
2	19:00:20	select currentBal('johndoe') "Balance" LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Query -> select currentBal('johndoe') "Balance";

Result Grid

	Balance
▶	1950.00

2. Function to get total Expense of a certain category.

```
delimiter //
create function expenseForCategory(in_category varchar(20))
returns decimal(10,2)
reads sql data
begin
    declare totalExpense decimal(10,2);
    select sum(amount) into totalExpense from expenses
    where categoryid = (select CategoryID from Category where CategoryName = in_category);
    return totalExpense;
end;
//
delimiter ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:10:19	create function expenseForCategory(in_category varchar(20)) returns decimal(10,2) reads sql data begin declare ...	0 row(s) affected	0.016 sec
2	19:10:22	select expenseForCategory('Food') as FoodExpense LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Query -> select expenseForCategory('Food') as FoodExpense;

Result Grid

	FoodExpense
▶	150.00

Stored Procedure:
A stored procedure is a set of pre-compiled Structured Query Languages (SQL), so it can be reused and shared by multiple programs. It can access or modify data in a database.

Procedure:

1. Procedure to show Account Details

```
delimiter //
create procedure showAccountDetails (
 IN in_username varchar(50)
 )
begin
    select u.Username,a.accountnumber, a.accountid, a.accounttype, a.balance
    from Account a
    join User u on a.UserID = u.UserID
    where u.Username = in_username;
end;
//
delimiter ;
```

```
53  18:29:39  create procedure showAccountDetails ( IN in_username varchar(50) ) begin select u.Username,a.accountnu...  0 row(s) affected  0.016 sec
54  18:29:41  call showAccountDetails('johndoe')  1 row(s) returned  0.000 sec / 0.000 sec
```

Query -> call showAccountDetails('johndoe');

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Username	accountnumber	accountid	accounttype	balance
johndoe	101	1	Savings	1400.00

2. Procedure to change Password of User

```
delimiter //
create procedure changePassword (
    in_username varchar(50),
    in_password varchar(50),
    new_password varchar(50)
)
begin
    declare existing_pass varchar(50);

    select password into existing_pass from account where userid = (
        select userid from user where username = in_username
    );
    if existing_pass is not null and existing_pass = in_password then
        UPDATE Account SET password = new_password where userid = (
        select userid from user where username = in_username
    );
    end if;
end;
//
delimiter ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	18:46:02	create procedure changePassword (in_username varchar(50), in_password varchar(50), new_password va...	0 row(s) affected	0.015 sec
2	18:46:04	call changePassword('johndoe','password1','newpassword')	1 row(s) affected	0.000 sec

Query -> call changePassword('johndoe','password1','newpassword');

AccountID	AccountType	Balance	AccountNumber	Password	UserID
1	Savings	1400.00	101	newpassword	1

Trigger:
A trigger is also a set of SQL statements in the database which automatically execute whenever any special event occurs in the database, like insert, delete, update, etc.

Triggers:

1. Trigger to update Balance in Accounts table after inserting new Income to maintain Consistency in database.

```
delimiter //
create trigger afterincomeinsert
after insert
on Income
for each row
begin
    declare newbalance DECIMAL(10,2);
    declare updateid INT;
    SET updateid = (SELECT accountid FROM records WHERE recordid = NEW.recordid);
    SET newbalance = (SELECT balance + NEW.amount FROM account WHERE accountid = updateid);
    update account SET balance = newbalance WHERE accountid = updateid;
end;
//
delimiter ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
21	13:55:08	create trigger afterincomeinsert after insert on Income for each row begin declare newbalance DECIMAL(10,...	0 row(s) affected	0.000 sec
22	13:55:15	insert into income values(11, 11, 400, '2024-05-16', 'Gift from family', 10)	1 row(s) affected	0.000 sec

Query -> insert into income values(11, 11, 400, '2024-05-16', 'Gift from family', 10);

AccountID	AccountType	Balance	AccountNumber	Password	UserID
1	Savings	1400.00	101	password1	1
NULL	NULL	NULL	NULL	NULL	NULL

Balance updated in account table.

2. Trigger to update Balance in Accounts table after inserting new Expense to maintain Consistency in database.

```
delimiter //
create trigger afterExpenseinsert
after insert
on Expenses
for each row
begin
    declare newbalance DECIMAL(10,2);
    declare updateid INT;
    SET updateid = (SELECT accountid FROM records WHERE recordid = NEW.recordid);
    SET newbalance = (SELECT balance - NEW.amount FROM account WHERE accountid = updateid);
    update account SET balance = newbalance WHERE accountid = updateid;
end;
//
delimiter ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
28	14:03:34	create trigger afterExpenseinsert after insert on Expenses for each row begin declare newbalance DECIMAL...	0 row(s) affected	0.016 sec

Query -> insert into expenses values(12,11,100, '2024-02-25', 'Groceries',1);

AccountID	AccountType	Balance	AccountNumber	Password	UserID
2	Checking	400.00	102	password2	2
NULL	NULL	NULL	NULL	NULL	NULL

Balance updated in account table.

References:

Tools Used:

1. MySQL :: Download MySQL Workbench
2. Flowchart Maker & Online Diagram Software **(Draw.io)**
3. canva.com

Research:

1. Software Architecture of Online Personal Finance Management System | Maharaj's Memoirs - Academia.edu
2. (PDF) DESIGN AND DEVELOPMENT OF PERSONAL FINANCE MANAGEMENT SYSTEM (researchgate.net)
3. Database Design in DBMS - GeeksforGeeks